of the working directory, the unmodified files, modified files and staged files.

Displaying differences

We can view the differences between different commits, or between the last commit and the modified files in the current directory, or staged files. To see the difference between the last commit and the changes made in the repo, use *git diff*. To see the difference between the last commit and the staging area (files added using *git add*, but not committed), use *git diff --staged*.

It is also possible to see the difference between a given commit ID and the current modified files with *git diff commit_id*. To see the difference between current files against a different branch, use *git diff branchname*. It is possible to see how many lines got added or removed using *git diff --stat*. To display the difference between the current repo and an *nth* old commit, use *git diff HEAD~N*.

Resetting or reverting changes

After making a change, you may later decide to remove it. The Git *reset* command is used to reset some changes. There are three different types of resetting—soft, hard and mixed. Let us look at each of them.

When you add some changes to the staging area using *git add*, and those changes haven't been committed, you can use *git reset* to remove those files/changes from the staging area. The actual changes made to the working copy of the file are unaffected. The default Git reset is equivalent to *git reset –mixed*.

If you want to reset the HEAD of the current branch to some commit, *git reset –soft COMMIT_ID* can be used. It will not destroy anything, but it will just reset the HEAD.

The *hard* option will do three things; it will clear the staging area, reset the HEAD to a recent HEAD or specified COMMIT_ID and delete the local file modifications.

Checking out

If you want to get files from a specified commit or branch, you can use *git checkout branchname*. You can also check out only a single file from a branch with *git checkout master filename*.

Cloning a remote repository

Any public Git repository can be cloned to a local repository using the *git clone* command, as *git clone repo_path*. Here, *repo_path* can be an HTTP URL, local UNIX-style path or SSH location.

Pushing local commits to a remote repository

You make changes to your local repository and also need to push the changes to the main repository frequently. You can push changes with *git push remote_repo branchname*.

If you created your local repo by cloning the main repo, the *remote_repo* name will be *origin*. You can push the changes to the master branch with *git push origin master*. If it is a fast-forward merge, it will succeed. If there are conflicts, you have to rebase your code and push it again.

Branches

In Git's workflow, branches are very important. You can check which branch you have currently checked out using the *git branch* command, which lists different branches and shows a * mark on the currently checked out branch.

You can create a new branch using *git branch new-topic-branch*. You can switch between branches using *git checkout branchname.* There is a shortcut to create and switch to the new branch:

git checkout -b newbranch.

A branch can be deleted using the *-D* option, such as *git branch -D new-topic-branch*.

Managing a remote repository

Often, you work on a local repository on your machine, but may need to sync with multiple remote branches. GitHub is a social code-hosting website that enables you to create remote repositories. Create a *github.com* account to play around and share your code.

You can list the different remote repositories that are linked to your local repository with *git remote*; add the –v switch for more verbose output. A remote repository can be linked to the current local Git repository with *git remote add repo_name repo_path*. For example:

```
$ git remote add my_test_repo git@github.com:slynux/coderepo.git
$ git remote #List remote repos for this repository
my_test_repo
```

All branches, including remote repo branches, can be listed with *git branch -a*. You can delete a branch from a remote repository with *git push repo_name :branch_name*. If the current repo is cloned from the remote repo, use *git push origin :branch_name*.

In this article, we have covered the essential concepts related to Git and the basic commands to work with it. My next article will cover more advanced features, including merges, rebasing, tagging, cherry-picking and an example workflow by integrating with GitHub. Happy hacking! END

By: Sarath Lakshman

The author is a software engineer as well as a hacktivist of Free and Open Source Software and passionate about UNIX like operating systems and design. He has recently authored the Linux Shell Scripting Cookbook, which gives insights on shell scripting through 119 recipes. He can be reached via his website *http://www.sarathlakshman.com*.